



Math Dungeon

Level up your math brain

Software Requirements Specification
Version 1.0

Project Name: Math Dungeon
Theme: Mathematical Educational Game
Category: Unity Game Foundry

Table of Contents

1.1 Background and Necessity	3
1.2 Proposed Solution	8
1.3 Scope of Project	11
1.4 Constraints	16
1.5 Application Architecture Diagram	18
1.6 Flowchart	21
1.7 Functional Requirements	22
1.8 Non-Functional Requirements	30
1.9 Data Dictionary	33
1.10 Interface Requirements.....	37
1.11 Project Submission Details	38

1.1 Background and Necessity

Mathematics is a foundational subject for logical reasoning, quantitative decision-making, and many areas of science and technology. However, repeated worksheet practice can become monotonous for learners, especially when the activity does not show how a



mathematical idea connects to an observable outcome. An educational game can address this engagement gap by placing mathematical thinking inside a context where the learner has a clear reason to solve a problem.

Game-based learning is particularly useful when the interaction is tied directly to the learning objective rather than presenting a separate quiz upon completion of gameplay. **Math Dungeon** is therefore conceived as a focused educational adventure in which mathematical reasoning is the main mechanism for opening routes, overcoming obstacles, and progressing through the dungeon. The project is intended to strengthen practice and confidence without turning the game into a conventional examination interface.

The necessity of the project arises from the requirement for a balanced learning experience in which entertainment supports rather than replaces the academic objective. The planned desktop build keeps the content focused on a limited set of mathematics topics and a small number of dungeon levels. This allows students to complete the project within the available development duration while still demonstrating meaningful Unity game development, AI, and Data-Oriented Technology Stack (DOTS) implementation.

Market Trends and Analysis

The market for educational software increasingly emphasizes interactive learning, short activity cycles, and immediate

feedback. Educational games can provide a controlled environment where learners practice a concept repeatedly while seeing a visible consequence for each successful decision.

- **Growth of Game-Based Learning:** Schools, parents, and training organizations increasingly use digital activities to supplement conventional teaching. Games are attractive because they can convert repeated practice into goal-oriented tasks with clear feedback and progression.
- **Demand for Mathematics Practice:** Mathematics remains a high-practice subject because fluency develops through repeated problem-solving. A focused game can provide structured practice for selected topics while maintaining a consistent difficulty curve and clear learning outcomes.
- **Preference for Interactive Feedback:** Learners benefit when they immediately know whether their reasoning is correct and can see a brief explanation of the result. **Math Dungeon** uses this principle to make each challenge informative rather than treating a wrong answer as a simple failure.

Requirement for Gamified Mathematical Learning

The central learning requirement of **Math Dungeon** is to make mathematical reasoning an active part of the adventure rather than an unrelated assessment. The player must understand the problem, choose a solution, and observe a direct gameplay consequence creating a clear link between the academic task and the game objective.

- **Concept Reinforcement:** Repeated challenges reinforce selected mathematics topics through practice while the game context gives each question a practical purpose, such as opening a route or disabling an obstacle.
- **Confidence Building:** A gradual difficulty curve and optional assistance allow players to make progress without being forced to solve advanced questions prior to the demonstration of the required foundation.
- **Immediate Understanding:** Correct and incorrect responses are followed by concise feedback, so the learner can connect the answer with the underlying mathematical idea prior to continuing the adventure.

Target Audience:

Math Dungeon is designed to serve multiple categories of learners and educational institutions. The game focuses on improving mathematical abilities while maintaining an entertaining gameplay experience.

Math Dungeon is designed to attract following audience segments:

1. **School Students:** Primary and secondary school students can use the game to strengthen their understanding of mathematical concepts taught in the classroom. The adventure-based gameplay encourages regular practice without making learning feel repetitive or stressful.
2. **Parents and Home Learners:** Parents looking for productive screen time can use **Math Dungeon** as a supplementary learning tool for their children. The game provides an enjoyable environment where students can practice mathematics independently while receiving immediate feedback on their progress.
3. **Teachers and Educational Institutions:** Teachers can incorporate the game into classroom activities or assign it as additional practice outside school hours. The structured levels and gradually increasing difficulty make it suitable for reinforcing concepts taught during mathematics lessons.

Figure 1 shows the target audience of end users and current deployment boundary of the game.

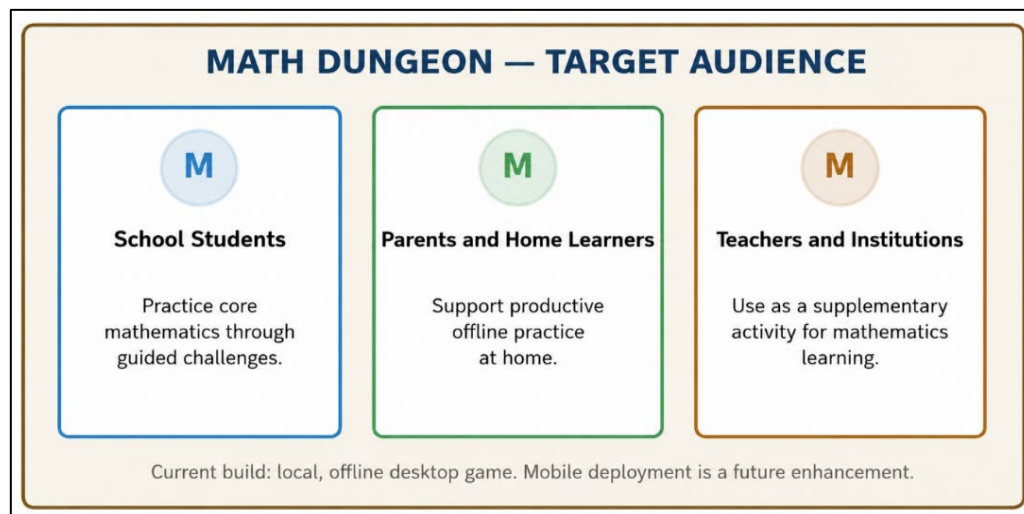


Figure 1: Target Audience and Current Deployment Boundary

Technological Feasibility

The project is technically feasible within Unity because the planned gameplay uses standard systems. Standard systems include character movement, scene management, UI, scripted mathematical validation, local JavaScript Object Notation (JSON) storage, and AI-based pursuit. The project does not depend on online services for its core loop, which reduces implementation and testing complexity.

- **Unity and C#:** Unity provides the runtime, physics, animation, UI, scene, and deployment systems required for the game while C# supports the mathematical logic, progression rules, AI integration, and save management.
- **Audio Pipeline:** Audacity and Firelight Multimedia Object Designer (FMOD) Studio support editing and runtime audio organization without requiring a large external content pipeline.
- **AI and DOTS:** The core build includes an AI pursuit behavior for dungeon enemies and a DOTS/Entity Component System (ECS)-based system for repeated entities such as collectibles. These components demonstrate AI and performance-oriented Unity development without making the project dependent on a large number of advanced systems.

Player Engagement and Retention

Engagement in **Math Dungeon** is based on a simple learning loop that includes the player exploring, encountering a mathematical problem, receiving feedback, and using the result to continue. This keeps the educational task directly connected to the game objective and avoids introducing unrelated reward mechanics that would significantly expand the implementation scope.

Figure 2 shows the core learning and gameplay loop.

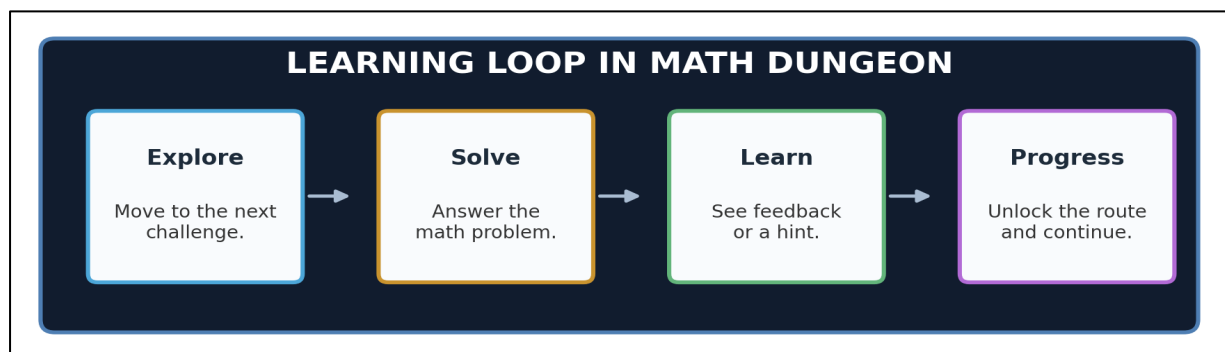


Figure 2: Core Learning and Gameplay Loop

Math Dungeon incorporates several key elements designed to keep players hooked, such as:

- **Enemy**: A single basic AI-controlled dungeon creature that patrols and chases the player within a defined area. It triggers a math challenge when it encounters the player, rather than engaging in conventional combat.
- **Progressive Difficulty**: Each dungeon introduces increasingly challenging mathematical concepts, allowing players to build confidence before encountering more complex problems. This gradual progression helps students improve their skills without becoming overwhelmed.
- **Reward System**: Players receive coins and unlockable content upon successful completion of mathematical challenges. These rewards provide positive reinforcement and motivate students to continue learning while tracking their academic progress.
- **Interactive Learning Environment**: Instead of presenting isolated mathematical questions, the game integrates problems into exploration, puzzles, and dungeon progression. This contextual learning approach improves understanding by

allowing players to apply mathematical concepts during gameplay.

- **Performance Feedback:** The game provides immediate feedback after each mathematical challenge by explaining correct and incorrect answers. This feature helps students identify mistakes, improve problem-solving strategies, and strengthen conceptual understanding through continuous practice.

1.2 Proposed Solution

Game Overview

TechnoWizards Learning Solutions aims to develop an engaging offline educational adventure game called **Math Dungeon**. The game is designed to help students strengthen their mathematical problem-solving abilities by combining adventure gameplay with progressively challenging mathematical puzzles. Instead of relying solely on textbooks and worksheets, players learn mathematical concepts by exploring dungeon environments, overcoming obstacles, and solving problems that directly influence their progression.

Math Dungeon is a focused educational puzzle-adventure game in which the player controls a young adventurer moving through three themed dungeons. Progress depends on solving mathematical challenges that are integrated into the environment. A correct solution can unlock a gate, activate a bridge, disable an obstacle, or weaken an AI pursuer. An incorrect response triggers a controlled setback and another opportunity to learn.

The project deliberately limits the number of dungeons and mathematical challenges so that the core experience can be implemented and tested within development hours. Each dungeon contains four mandatory mathematical challenges, giving a total of twelve core challenges for the complete game. Optional decorative rooms and cosmetic content can be added only after the stability of all mandatory requirements.

The mandatory mathematical curriculum consists of topic groups, including arithmetic, fractions and percentages, algebra, geometry, and logical reasoning with patterns. Each dungeon

emphasizes one or more topics while the final dungeon acts as an integrated review. Question formats include multiple-choice, numerical input, and pattern or relationship matching so that the learner is not restricted to one style of response.

Level and Learning Structure

- **Dungeon 1 - Arithmetic Foundations:** Four challenges focus on addition, subtraction, multiplication, division, comparisons, basic fraction operations, simple percentage applications, and order of operations. The purpose is to establish fluency prior to the player encountering multi-step problems.
- **Dungeon 2 - Algebraic Thinking and Geometry:** Four challenges introduce unknown values, simple linear equations, expressions, substitution, perimeter, area, angle relationships, and simple shape properties.. The challenge design emphasizes reasoning about relationships rather than memorizing isolated formulas.
- **Dungeon 3 - Logic and Integrated Review:** Four challenges combine patterns, numerical relationships, and selected concepts from earlier dungeons. The final dungeon is designed as a review rather than a new high-complexity curriculum block.

Key Features of Math Dungeon

- **Integrated Mathematical Progression:** Mathematical challenges are seamlessly embedded into core gameplay rather than presented as separate quizzes. Players encounter problems naturally as environmental barriers, such as locked dungeon gates, active traps, missing bridges, or chest locks. Progressing through the dungeon requires applying topic-specific mathematical concepts, ensuring that academic problem-solving is directly tied to gameplay progression.
- **AI Pursuit Encounter:** A dungeon enemy uses AI to detect, pursue, and pressure the player. When the player reaches a



designated challenge zone, the encounter is converted into a mathematical decision rather than a conventional combat sequence.

- **DOTS-Based Entity Management:** DOTS/ECS manages repeated lightweight entities such as collectible objects. This provides a practical use of the DOTS stack without forcing the entire project into ECS architecture.
- **Basic Hint and Learning Assistance:** The hint system provides limited guidance rather than an instant answer. The assistance is used when the learner is stuck and is designed to keep problem-solving active.
- **Local Progress and Assessment:** The game records the current local profile, completed challenges, accuracy, and unlocked dungeons on the same device. No online account or cloud synchronization is required.
- **Replay and Review:** Completed dungeons can be replayed so that learners can practice weak topics again. Replay improves the educational value without requiring additional story content or a larger number of levels.
- **Accessible User Interface:**
 - The interface is designed with simplicity and readability in mind, ensuring that players can easily navigate menus, mathematical questions, and progress reports. Important information remains clearly visible without distracting from gameplay.
 - Consistent layouts, readable fonts, and intuitive icons create an accessible experience suitable for younger learners as well as experienced players.
- **Performance Optimization:**
 - The game is optimized to provide smooth gameplay across supported desktop platforms. Efficient resource management minimizes loading times, maintains stable frame rates, and ensures responsive controls even during complex puzzle sequences.
 - Optimization allows players to focus on exploration and learning without experiencing technical interruptions that could reduce educational effectiveness.

1.3 Scope of Project

The scope of **Math Dungeon** is to develop a small, offline, single-player educational mathematics game that can be implemented and tested within the development timeframe. The project focuses on the core learning gameplay loop in which the player explores a dungeon, encounters an enemy, solves a mathematics challenge, and continues based on the result.



The game contains **three dungeon areas** with **four mathematics challenges in each dungeon**, resulting in **twelve predefined challenges** in total. The challenges use **three selected mathematics question types** with the difficulty increasing between the three dungeon areas.

The player can move through the dungeon and encounter a **single basic enemy type**. When the player enters the enemy's detection or proximity range, player movement can be temporarily paused and a mathematics challenge is displayed. A correct answer allows the player to defeat or dispel the enemy and continue. An incorrect answer reduces player's health by one heart (starting from a fixed pool of three hearts) and provides a predefined conceptual hint before allowing player to retry/retreat.

The game includes a simple health system, basic coin rewards, predefined hints, dungeon objectives, and local progress saving. The gameplay follows a **linear progression** from the first dungeon to the third dungeon and concludes after the required challenges have been completed.

The project uses Unity as the game engine and focuses on implementing the essential educational and gameplay mechanics rather than a large-scale progression or content system. Features that require substantial additional development are not part of the mandatory implementation. These include dynamic question

generation, multiple enemy types, complex AI, inventory management, achievements, multiplayer functionality, and advanced machine-learning behavior. They are out of scope.

The detailed scope of the project is explained as follows:

Core Gameplay Mechanics

1. Standard Player Controller:

Simple WASD/Keyboard movement with basic interactions (pressing 'E' near terminals)

- **NavMesh Chase Enemy:** A simple patrol/chase enemy powered by a standard C# script and Unity's built-in NavMeshAgent. If the enemy touches the player, it triggers a temporary math penalty screen or health deduction.
- **Local Save File:** Automatic progression tracking saved locally using a single-file JSON system.

2. Animation (Optional):

- **Character Movement:** The player can perform smooth walking, running, jumping, and interaction animations. Responsive movement animations improve gameplay quality and provide a better learning experience.
- **Interactive Environment:** Doors, bridges, moving platforms, and magical barriers contain interactive animations that respond to player actions. These environmental animations make the dungeon feel alive and interactive.
- **Visual Effects:** Particle effects such as glowing portals, coin collection and puzzle completion provide visual feedback whenever the player successfully completes mathematical challenges.

Mathematical Gameplay and Progressive Learning

1. Level Design:

- **Structured Learning Progression:** Each dungeon level introduces mathematical concepts in a gradual manner. Early levels focus on basic arithmetic, while later stages introduce multiplication, division, fractions, percentages, algebra, and logical reasoning to continuously improve the player's skills.
- **Unique Dungeon Layouts:** Each dungeon can have a different layout containing branching paths and puzzle chambers. This

variety encourages exploration and prevents gameplay from becoming repetitive.

- **Balanced Difficulty:** The complexity of mathematical questions and dungeon challenges increases according to the player's progress. This balanced progression keeps the game challenging without becoming frustrating for new learners.

2. Mathematical Challenges:

- **Arithmetic Challenges:** Players solve addition, subtraction, multiplication, and division questions to unlock doors, activate bridges, defeat enemies, or collect important resources required for progression.
- **Logical Reasoning Puzzles:** Certain dungeon sections contain logic-based puzzles where players must identify patterns, sequences, or correct mathematical relationships. These activities improve critical thinking in addition to calculation skills.

Responsive Controls and User Interaction

1. Control System:

- **Simple Player Controls:** The game uses intuitive keyboard, mouse, or touch controls that allow players to move, interact with objects, answer mathematical questions, and navigate menus comfortably. Easy controls ensure accessibility for players of different age groups.

2. Performance

Optimization:

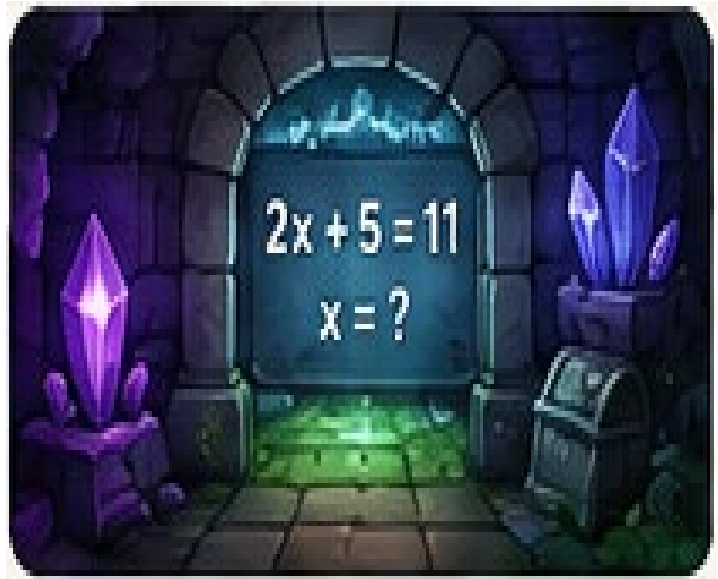
➤ **Responsive**

Gameplay: Player inputs can be processed with minimal delay to ensure smooth movement and accurate interactions during mathematical challenges.

Responsive

gameplay helps maintain concentration throughout the learning experience.

- **Device Compatibility:** The application can be optimized for Windows and macOS. Performance optimization ensures consistent gameplay across different hardware configurations.



Reward and Progression System

1. Experience System:

- **Knowledge-Based Progress:** Players earn experience points after successfully completing mathematical puzzles and dungeon objectives. Accumulated experience unlocks new levels, advanced mathematical topics, and additional gameplay content.
- **Player Progress Tracking:** The game continuously monitors completed levels, solved questions, earned rewards, and overall learning progress. This allows players to observe their improvement over time.

2. Reward Mechanism:

- **Coins:** Correct answers reward players with coins.

3. Educational Assistance System:

- **Learning Support:**
 - **Hint System:** Players request hints when facing difficult mathematical problems. Instead of directly revealing the answer, the game provides useful guidance that encourages independent problem-solving.
 - **Solution Explanation:** After completing selected mathematical challenges, the game displays a short explanation describing how the correct answer was obtained. This reinforces learning rather than simple memorization.



4. User Interface Design:

- **Main Menu:** The main menu provides access to New Game, Continue, Level Selection, Settings, Rewards, Help, and Exit options. The interface is designed to be clean, organized, and easy to navigate.
- **In-Game Heads-Up Display (HUD):** The HUD presents important gameplay information that includes player health, current objective, mathematical score, elapsed time, collected coins, and dungeon progress. The layout ensures that essential information remains visible without distracting the player.
- **Settings Menu:** The **Settings** menu allows players to modify audio, graphics quality, control preferences, and accessibility options. Personalized settings improve comfort for different users.

5. Save and Progress Management:

- **Save System:** The application automatically saves player progress after completing important checkpoints and dungeon levels. Automatic saving minimizes the risk of losing progress unexpectedly.

- **Load System:** Players can continue their previously saved adventure from the last completed checkpoint. The loading system enables users to resume learning without restarting completed stages.

1.4 Constraints

The development of **Math Dungeon** involves several technical, operational, and project-related constraints that must be carefully managed throughout the software development lifecycle. These constraints define the limitations under which the project must be designed, developed, tested, and deployed.

Addressing these challenges effectively ensures that the game remains educational, engaging, and technically reliable.



Following constraints can be applied to the game:

- **Limited Hardware Performance:** **Math Dungeon** is designed to run on a variety of desktops with different hardware specifications. Low-end devices experience reduced performance when rendering complex environments, particle effects, or multiple enemy characters simultaneously. Therefore, graphical assets and game logic must be optimized to maintain stable frame rates.
- **Device Compatibility:** The game is intended to support offline desktop/laptop devices on Windows and macOS platforms. Differences in operating systems, screen resolutions, input methods, and hardware capabilities require platform-specific testing and optimization to ensure consistent gameplay across all supported devices.
- **Memory Management:** Educational games often include numerous visual assets, audio files, animations, and user interface elements. Poor memory management can increase loading times and lead to application crashes. Efficient asset loading and memory optimization techniques are necessary to provide a smooth user experience.
- **Performance Optimization:** The game includes animated characters, mathematical puzzles, enemy AI, particle effects, and interactive environments. These features consume

processing resources, making optimization essential to prevent lag and ensure responsive gameplay, especially on devices with limited computational power.

- **Third-Party Dependency: Math Dungeon** relies on external tools and libraries such as Unity, Visual Studio, Blender, Photoshop, and FMOD Studio. Updates or compatibility issues within these tools require modifications to the project and could temporarily delay development activities.
- **Curriculum Accuracy:** All mathematical questions and problem-solving activities must be factually correct and appropriate for the selected educational level. Incorrect mathematical content would negatively impact the learning objectives of the game.
- **Difficulty Balancing:** Questions should gradually increase in complexity as players progress through the dungeon. If challenges become too difficult too quickly, players lose motivation while overly simple questions can reduce engagement and educational value.
- **Learning and Entertainment Balance:** The game must successfully combine education with entertainment. Excessive educational content makes gameplay feel like a traditional examination, whereas focusing only on gameplay could reduce the effectiveness of the learning experience.
- **Screen Adaptability:** The interface must automatically adjust to different screen sizes and resolutions without affecting readability or gameplay. Buttons, mathematical equations, and menus should remain properly aligned across supported devices.
- **Accessibility:** Fonts, colors, icons, and interactive elements should be easy to understand and clearly visible. The interface should minimize confusion and allow players to navigate the game without requiring extensive instructions.
- **Save Data Protection:** Player progress, completed levels, scores, and unlocked content should be securely stored to prevent accidental data corruption or unauthorized modification.
- **Data Integrity:** Game files and saved records should be validated during loading to prevent corrupted or incomplete data from affecting gameplay or causing unexpected application errors.

- **Asset Availability:** Creating high-quality 2D or 3D models, educational graphics, animations, and sound effects requires significant development effort. Limited asset availability requires developers to use optimized or reusable resources while maintaining visual consistency.
- **Budget Limitations:** The project primarily utilizes publicly available development tools and educational licenses. Budget restrictions limit access to premium assets, plugins, or third-party software during development.
- **Human Resources:** The project's quality depends on the skills and experience of the development team. Limited expertise in specialized areas such as AI, advanced animations, or educational content design requires additional research and learning during implementation.

1.5 Application Architecture Diagram

The **Math Dungeon** application follows a decoupled three-tier layered architecture separating presentation, core gameplay logic, and data persistence. This separation ensures modularity, enabling developers to update educational content or UI elements without altering core game loop systems.

Figure 3 shows the application architecture for the game.

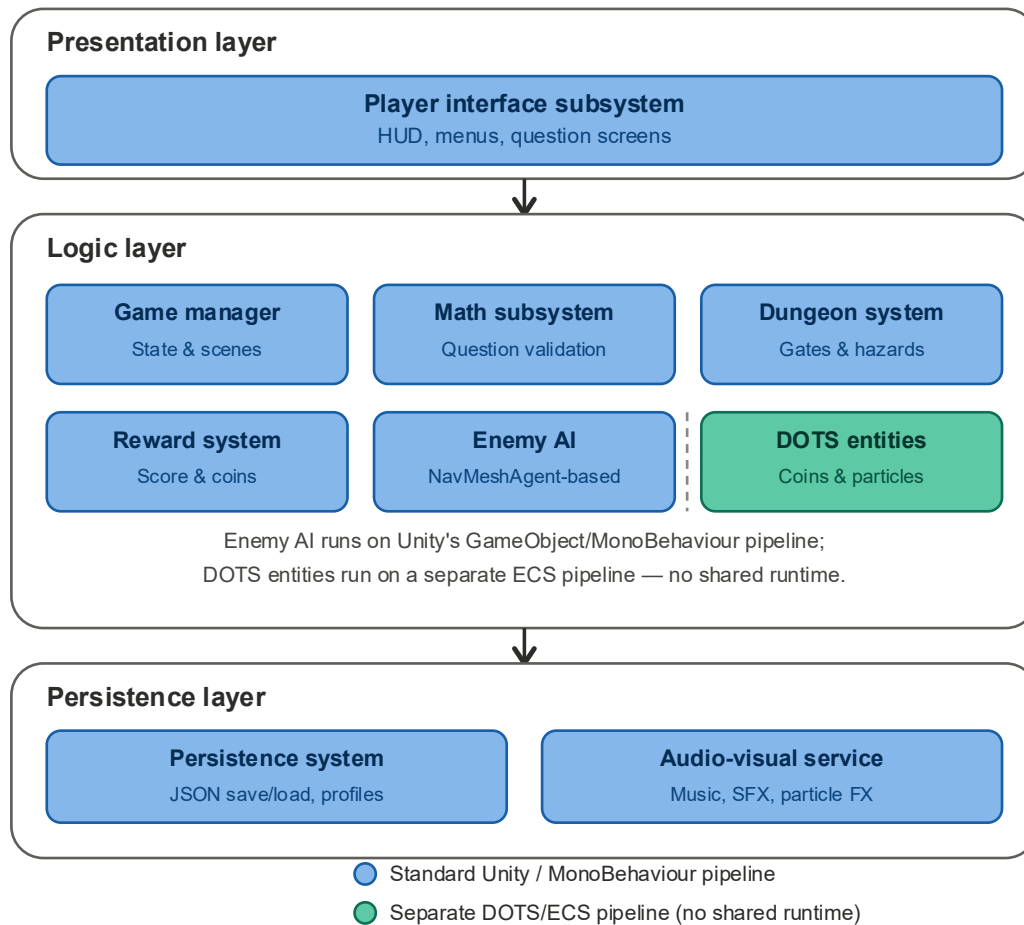


Figure 3: Application Architecture for the Game

1. Presentation Layer (UI/UX)

- **Player Interface Subsystem:** Renders real-time gameplay, HUD displays, question dialogs, and receives user input.

2. Logic Layer

- **Game Manager:** Controls application state, scene transitions, and global session conditions.
- **Mathematics Subsystem:** Handles predefined question selection, answer verification, and hint calculation.
- **Dungeon Environment Subsystem:** Manages level hazards, interactive gates, and entity spawning.
- **Enemy AI Controller:** A standard GameObject-based component using Unity's built-in NavMeshAgent to handle enemy patrol and chase behavior. Runs on the conventional MonoBehaviour pipeline, independent of the ECS world.
- **DOTS Entity Manager:** A separate ECS pipeline that manages lightweight, high-count world entities such as collectible coins and particle-driven visual effects. Operates independently of the GameObject-based enemy AI; the two systems do not share entities or a runtime loop.
- **Reward System:** Processes score tracking and currency calculation after puzzle resolution.

3. Persistence Layer

- **Persistence System:** Manages read/write operations for player progression, profile settings, and unlocked content using local JSON serialization.
- **Audio-Visual Service:** Manages background music, dynamic audio triggers via FMOD/Unity Audio, and particle effects.

1.6 Flowchart

Figure 4 shows the flowchart for the application.

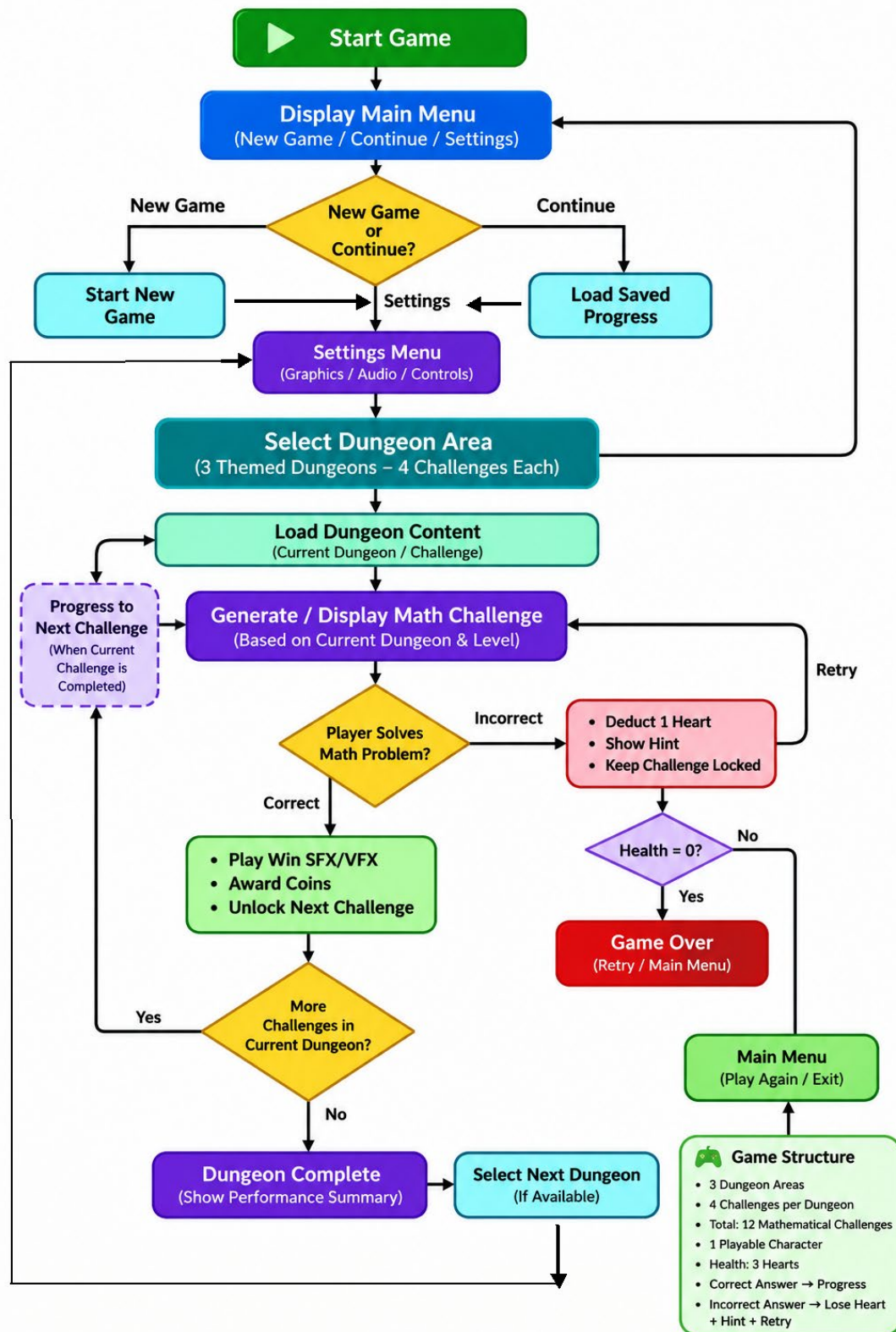


Figure 4: Flowchart for the Application

The flowchart shows the logical sequence of operations performed by the game from launch to completion. It provides a clear representation of how the player interacts with different game modules, including level selection, gameplay, puzzle solving, score calculation, and game completion. The flow ensures that each stage of the game is connected systematically, allowing smooth progression and minimizing logical errors during development.

The flowchart also serves as a guide for developers to understand the interaction between different modules of the application. It helps identify decision points, player actions, and possible outcomes, making implementation and testing more organized and efficient.

1.7 Functional Requirements

Functional requirements define the core features and system behaviors that the **Math Dungeon** game must implement to provide an engaging educational experience. These requirements describe how the player interacts with the game, how mathematical challenges are presented, how progress is tracked, and how rewards are provided.

Each requirement contributes to create an enjoyable learning environment while maintaining balanced gameplay and educational value.

1. Local Profile Creation and Player Progression: The game allows players to create and maintain a local player profile on the device to save their progress and learning statistics.

As **Math Dungeon** is designed strictly as an offline, single-device application, player profiles are saved locally using device JSON storage without requiring online registration, user accounts, or Internet connectivity. This local profile enables the game to provide a personalized experience by persisting completed levels, unlocked content, and performance records directly on the local machine.

Functional Specifications:

- **Profile Creation:** Players can create a new profile by entering a username and selecting an avatar. The profile acts as the player's identity throughout the game.
 - **Profile Management:** Players can have the option to edit their profile details, change avatars, and manage saved game progress without affecting other player profiles.
 - **Progress Storage:** The game should automatically save completed levels, earned rewards, unlocked dungeons, and mathematical performance for future gameplay sessions.
- 2. Main Menu Navigation:** The main menu serves as the central navigation hub of the application. It provides access to all major game functions and allows players to start, continue, or customize their gameplay experience.

Figure 5 shows the flow for profile selection.

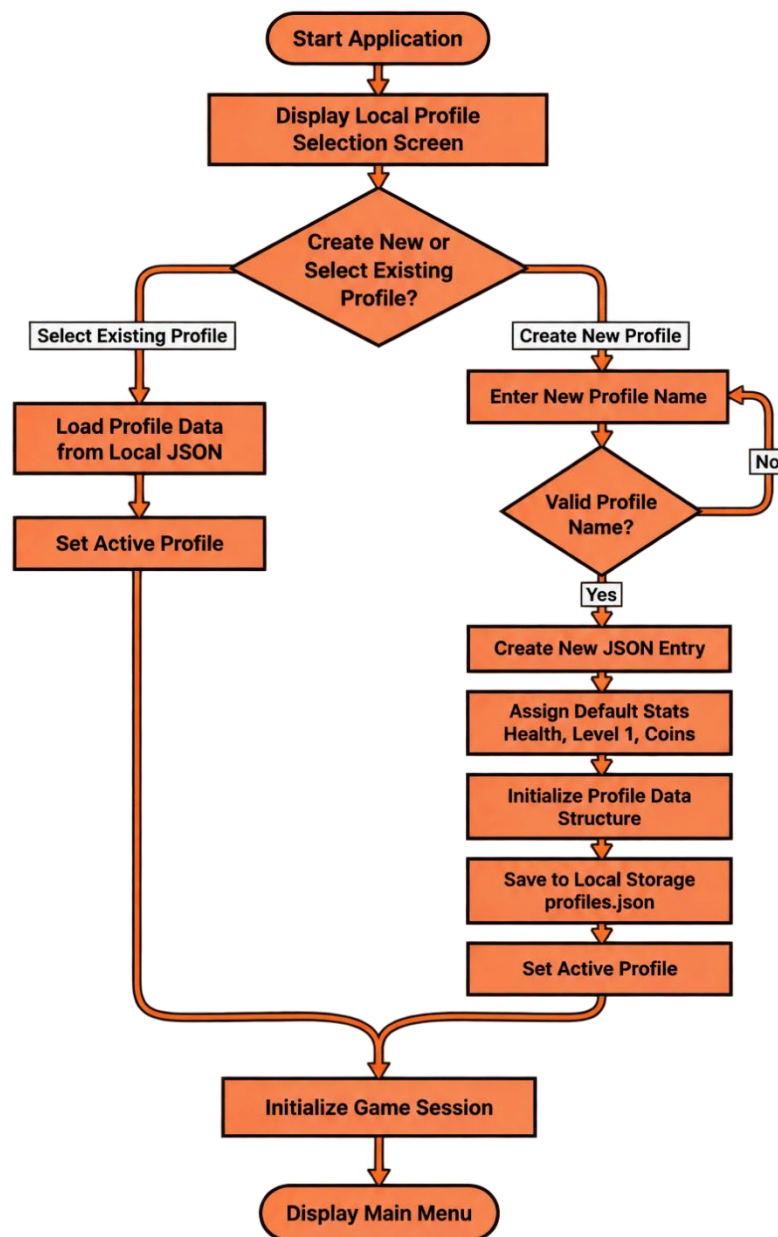


Figure 5: Profile Selection Flow on Game Startup

Functional Specifications:

- **New Game:** The player shall be able to begin a new adventure from the first dungeon while resetting previous gameplay data only when confirmed.
- **Continue Game:** The game shall load the player's most recently saved progress, allowing uninterrupted continuation of previously completed sessions.

- **Settings Access:** Players shall be able to customize game settings, including audio, graphics, controls, and accessibility options before or during gameplay.

3. Dungeon Level Management: The game shall organize gameplay into multiple dungeon levels with gradually increasing mathematical difficulty. Each dungeon introduces new educational concepts while maintaining an engaging adventure experience.

Functional Specifications:

- **Sequential Progression:** Players shall unlock the next dungeon only after successfully completing the current level's objectives and mathematical challenges.
- **Difficulty Progression:** Each new dungeon shall introduce more complex mathematical concepts, ensuring gradual improvement in problem-solving abilities.
- **Checkpoint System:** The game shall provide checkpoints within larger levels so that players can resume from important milestones instead of restarting the entire dungeon.

4. Mathematical Challenge System: Mathematical puzzles are the primary educational component of the game. Each puzzle must evaluate the player's mathematical understanding before allowing progression.

Functional Specifications:

- **Problem Selection:** The game shall present the predefined mathematical challenge associated with the current dungeon and progression stage.
- **Answer Validation:** The player's answer shall be verified immediately, providing instant feedback indicating whether the solution is correct or incorrect.
- **Multiple Question Types:** The game shall support different question formats such as multiple-choice questions, numerical input, and pattern-solving exercises to maintain variety.

5. Character Movement and Exploration: Players shall control their character to explore dungeon environments, collect resources, and interact with educational objects. Smooth movement is essential for maintaining enjoyable gameplay.

Functional Specifications:

- **Movement Controls:** The character shall support walking, running, jumping, and interaction using keyboard or touchscreen controls depending on the platform.
- **Object Interaction:** Interactive objects such as doors, switches, and puzzle terminals shall respond appropriately when activated by the player.

- 6. Enemy Encounter System:** Enemies provide additional gameplay challenge by preventing unrestricted exploration. Players must solve mathematical questions or complete specific actions to overcome enemy encounters.

Functional Specifications:

- **Enemy Detection:** Enemies shall detect nearby players using predefined detection ranges and initiate encounters when the player enters their area.
- **Educational Combat:** Instead of traditional combat, certain enemies shall require players to correctly answer mathematical questions before they can continue.
- **Difficulty Progression:** Challenges in the second and third dungeon use more complex mathematical concepts than those in the first dungeon.
- **Health System:** The player shall begin each dungeon with a fixed pool of three hearts. An incorrect challenge answer shall deduct one heart. Reaching 0 hearts shall trigger a Game Over state, offering the player the option to retry the current challenge zone or return to the Main Menu.

- 7. Hint and Learning Assistance:** The game shall assist players who experience difficulty without immediately providing the correct solution. The objective is to encourage independent learning while reducing frustration.

Functional Specifications:

- **Hint System:** Players shall receive optional hints explaining the mathematical concept required to solve the current problem.
- **Step-by-Step Guidance:** Complex mathematical problems shall provide simplified intermediate steps after multiple unsuccessful attempts.

- **Practice Opportunity:** Players shall be allowed to practice similar mathematical problems before attempting the original challenge again.

8. Reward System: The reward system motivates players to continue learning by recognizing academic improvement and gameplay accomplishments. Rewards increase player engagement while reinforcing educational success.



Functional Specifications:

- **Coin Rewards:** Players shall receive coins after successfully solving mathematical challenges and completing dungeon objectives.
- 9. Scoring and Performance Evaluation:** The scoring system evaluates both educational performance and gameplay efficiency. It encourages players to improve their mathematical accuracy rather than relying solely on speed.

Functional Specifications:

- **Score Calculation:** Scores shall be calculated using factors such as correct answers, completion time, collected rewards, and overall accuracy.
- **Performance Summary:** At the end of each level, the game shall display detailed statistics showing solved questions, incorrect attempts, completion percentage, and earned rewards.
- **Learning Progress Report:** The game shall maintain cumulative reports illustrating improvements in mathematical skills across multiple gameplay sessions.
- **Completion Time Tracking:** The game shall record elapsed time from the start of a dungeon to its completion, for use in score calculation and the performance summary. This is a tracked metric only – the game does not impose a time limit, countdown, or time-based penalty; players may take as long as required to solve each challenge.

10. Save and Load System: The save system ensures that players do not lose progress during gameplay. It allows convenient continuation of learning without restarting completed content.

Functional Specifications:

- **Automatic Saving:** The game shall automatically save progress after major milestones such as level completion, and checkpoint activation.
- **Manual Saving:** Players shall also have the option to manually save their current progress whenever permitted by the game.
- **Saved Game Loading:** Previously saved game data shall be loaded accurately while preserving player statistics and unlocked levels.

11. Audio and Visual Feedback: The application shall provide immediate visual and audio feedback to improve user interaction and reinforce learning outcomes.

Functional Specifications:

- **Correct Answer Feedback:** Correct solutions shall trigger positive animations, sound effects, and reward notifications to encourage continued participation.
- **Incorrect Answer Feedback:** Incorrect answers shall generate constructive feedback that guides the player without creating frustration or discouragement.
- **Interactive Notifications:** The game shall display notifications for rewards, newly unlocked content, completed objectives, and important gameplay events.

12. Pause and Settings Management: Players shall be able to temporarily suspend gameplay and modify system settings without losing progress. This functionality improves usability and accessibility.

Functional Specifications:

- **Pause Function:** Players shall pause gameplay at any time during exploration while preserving the current game state.
- **Settings Modification:** Players shall adjust audio levels, graphics quality, and control sensitivity from the pause menu.
- **Resume Gameplay:** Selecting the resume option shall immediately return players to their previous gameplay state without restarting the level.

13. Game Completion and Progress Summary: Upon finishing the final dungeon, the application shall present a comprehensive summary of the player's educational learning and gameplay accomplishments.

Functional Specifications:

- **Final Statistics:** The game shall display total levels completed, mathematical accuracy, rewards earned, total score, and total gameplay duration.
- **Replay Option:** After completing the game, players shall have the option to replay previously completed dungeons to improve scores, practice mathematical concepts, and unlock remaining rewards.

1.8 Non-Functional Requirements

Non-functional requirements define the quality attributes of the **Math Dungeon** game. Unlike functional requirements, they do not specify what the game does, but describe how well the game should perform. These requirements ensure that the game is reliable, responsive, secure, user-friendly, and compatible with different systems. Meeting these requirements improves the overall gameplay experience and maintains player satisfaction throughout the educational journey.

Following are the non-functional requirements that have to be considered while creating the game application:

- 1. Performance Requirements:** The game must provide a smooth and responsive experience across all supported platforms. Performance optimization ensures that educational activities and gameplay mechanics run without noticeable delays or interruptions.
 - **Fast Game Loading:** The application should load quickly from the main menu to gameplay. Fast loading minimizes waiting time and allows players to begin solving mathematical challenges without unnecessary delays.
 - **Smooth Gameplay:** Character movement, enemy interactions, puzzle generation, and mathematical calculations should execute smoothly without frame drops. The game should maintain stable performance even during complex dungeon encounters.
 - **Optimized Resource Usage:** The game should efficiently utilize Central Processing Unit (CPU), Graphics Processing Unit (GPU), and memory resources to avoid excessive system load. Proper optimization allows the game to perform consistently on both minimum and recommended hardware specifications.
- 2. Reliability Requirements:** Reliability ensures that the application functions correctly during extended gameplay sessions and protects player progress from unexpected failures.
 - **Stable Game Execution:** The application should operate continuously without unexpected crashes or freezing during gameplay. Stability is essential for maintaining player engagement and preventing frustration.

- **Progress Preservation:** Player completed levels, collected rewards, and unlocked content should be stored accurately. In the event of an unexpected shutdown, previously saved progress should remain unaffected.
 - **Error Recovery:** If minor runtime errors occur, the application should recover gracefully without forcing the player to restart the entire game. Clear notifications should be displayed whenever recovery is possible.
- 3. Usability Requirements:** Usability focuses on making the game easy to understand and enjoyable for players of different age groups and mathematical skill levels.
- **User-Friendly Interface:** Menus, buttons, and gameplay screens should be organized clearly so that players can easily navigate between different game features without confusion.
 - **Easy Learning Curve:** New players should understand the gameplay through simple tutorials and guided instructions. Mathematical concepts should be introduced gradually to prevent overwhelming beginners.
 - **Accessibility Features:** The game should provide adjustable audio, display, and control settings. These options allow players with different preferences and abilities to enjoy the game comfortably.
- 4. Security Requirements:** Security requirements ensure that player information and game data remain protected from unauthorized access or modification.
- **Secure Save Files:** Game progress should be stored securely to prevent accidental corruption or unauthorized editing of player data.
 - **Data Protection:** Any personal information collected, such as player profiles or usernames, should be stored securely and used only for game-related purposes.
- 5. Compatibility Requirements:** Compatibility ensures that the game functions consistently across different operating systems and hardware configurations.

6. Maintainability Requirements: Maintainability ensures that developers can efficiently improve, update, and fix the application after deployment.

- **Modular Development:** The game should be developed using modular scripts and reusable components. This approach simplifies future enhancements and minimizes development effort.
- **Easy Bug Fixing:** Source code should be well-organized and documented so that developers can quickly identify and resolve issues without affecting unrelated systems.
- **Feature Expansion:** The project architecture should support the addition of new mathematical topics, dungeon levels, enemies, and educational activities without requiring major code restructuring.

7. Scalability Requirements: Scalability allows the application to grow with future educational content and gameplay features.

- **Additional Levels:** The game should allow developers to introduce new dungeon stages and mathematical challenges without impacting existing game functionality.
- **New Mathematical Topics:** Future updates should support additional educational content such as algebra, geometry, fractions, decimals, or advanced arithmetic while maintaining the existing learning framework.
- **Extended Reward System:** The reward system should accommodate new collectibles and unlockable content as the game expands.

8. Availability Requirements: Availability ensures that players can access and use the application whenever required.

- **Offline Gameplay:** The complete educational gameplay experience should remain available without requiring a continuous Internet connection. Players should be able to practice mathematics anywhere at their convenience.
- **Continuous Accessibility:** The application should remain available throughout normal gameplay sessions without unnecessary interruptions caused by background processes or temporary system events.

- **Save Accessibility:** Previously saved game data should be accessible whenever the player resumes the game, allowing uninterrupted continuation of educational progress.



These are the bare minimum expectations from the project. It is a must to implement the FUNCTIONAL and NON-FUNCTIONAL requirements given in this SRS. Once they are complete, you can use your own creativity and imagination to add more

features if required.

1.9 Data Dictionary

A sample data dictionary is shown here to define the JSON structures referenced throughout the document.

PlayerProfile

Field	Type	Description	Example
profileId	string	Unique identifier generated at profile creation	"p_0001"
username	string	Player-entered display name	"Alex"
avatarId	string	Selected avatar identifier	"avatar_03"
createdAt	string (ISO date)	Profile creation timestamp	"2026-08-01"
hearts	integer	Current health, 0–3	3
coins	integer	Total coins earned	120
unlockedDungeons	array of integer	Dungeon IDs the	[1, 2]

Field	Type	Description	Example
		player has access to	
dungeonProgress	array of DungeonProgresses	Per-dungeon completion state (see below)	—
lastCheckpoint	string	Identifier of the most recent saved checkpoint	"d2_c3"

DungeonProgress (one entry per dungeon, nested in PlayerProfile)

Field	Type	Description	Example
dungeonId	integer	1, 2, or 3	2
completed	boolean	Whether all 4 challenges are solved	false
challengesSolved	array of integer	IDs of challenges answered correctly	[201, 202]
attemptsPerChallenge	object (challengeId → integer)	Incorrect-attempt count per challenge, used for hint step-up (FR7)	{"203": 2}
elapsedTimeSeconds	integer	Time spent in this dungeon, for scoring (FR9)	340

Challenge (predefined question data — read-only content, not player-generated)

Field	Type	Description	Example
challengeId	integer	Unique challenge identifier	203
dungeonId	integer	Which dungeon this belongs to	2
topic	string	Curriculum topic tag	"linear equations"
questionType	enum	"multiple_choice" "numerical_input" "pattern_match"	"numerical_input"
promptText	string	The question shown to the player	" $5x + 7 = 22$, $x = ?$ "
options	array of string (nullable)	Answer choices, used only if questionType is multiple_choice	null
correctAnswer	string	The validated correct response	"3"
hintText	string	Conceptual hint shown after an incorrect attempt (FR7)	"Subtract 7 from both sides first."
explanationText	string	Shown after correct completion (per Educational Assistance System)	" $5x = 15$, so $x = 3$."

PerformanceRecord

Field	Type	Description	Example
sessionId	string	Unique play-session identifier	"s_0007"
profileId	string	Links back to PlayerProfile	"p_0001"
correctAnswers	integer	Correct answers this session	9
incorrectAttempts	integer	Total incorrect attempts this session	4
accuracyPercent	number	$\text{correctAnswers} / \text{totalAttempts} \times 100$	69.2
coinsEarned	integer	Coins earned this session	40
totalScore	integer	Computed score (FR9: accuracy + completion time + rewards)	860
completedAt	string (ISO date)	Session end timestamp	"2026-08-28"

1.10 Interface Requirements

Hardware Requirements

Windows Machines:

- Windows 10 or Windows 11, 64-bit or higher
- Intel i5/i7 or higher; AMD Ryzen 5 or higher
- 16 GB RAM or higher
- NVIDIA GTX 1660 or higher/AMD equivalent with 4 GB Memory or higher
- 1080 p or higher resolution monitor
- 1 TB SSD
- 4 GB graphics card (DirectX 11 or higher)
- Mouse and Keyboard

MacOS:

- macOS 10.15 or higher
- Intel Core i5/i7 or Apple M1 chip
- 16 GB RAM or higher
- Radeon Pro 560 with 4 GB Memory or higher
- 1080 p or higher resolution monitor
- 1 TB SSD
- 4 GB graphics card or Apple M1 integrated GPU
- Mouse and Keyboard

Software Requirements

- **Game Engine:** Unity - version 2020.3 or higher for advanced graphics and physics simulations
- **Unity Hub:** <https://unity3d.com/get-unity/download>
- **Operating System:** Windows 10, Windows 11, macOS 10.15 or higher
- **IDE:** Visual Studio 2019 or higher
- **Graphics and Animation:** Blender (latest version) and Adobe Photoshop (latest version)
- **Audio:** Audacity (latest version) and FMOD Studio (latest version)
- **Version Control:** Git (latest version), GitHub, or GitLab
- **Data Storage:** JSON/Other

1.11 Project Submission Details

Design, build, and submit the project along with a complete project report that includes:

- 1. Problem Definition:** Clearly state the problem addressed by **Math Dungeon**.
- 2. Design Specifications:** Detailed design documents and diagrams (such as flowcharts and data flow diagrams).
- 3. Installation Instructions:** Step-by-step guide for installing **Math Dungeon**.
- 4. User Credentials:** Share the test user accounts for review.

Project Files: Include project files (source code), Unity packages, and any additional resources required. Submit this as a consolidated zip file.

Ensure that all documentation is thorough and clear, providing a comprehensive overview of the game's functionality, architecture, and usage.



Documentation should not contain source code.

Submit a video (.mp4 file) demonstrating the working of the game application, including all the functionalities of the project. This is MANDATORY.

~~ End of the Document~~